Software Testing

Property-based Testing with Hypothesis

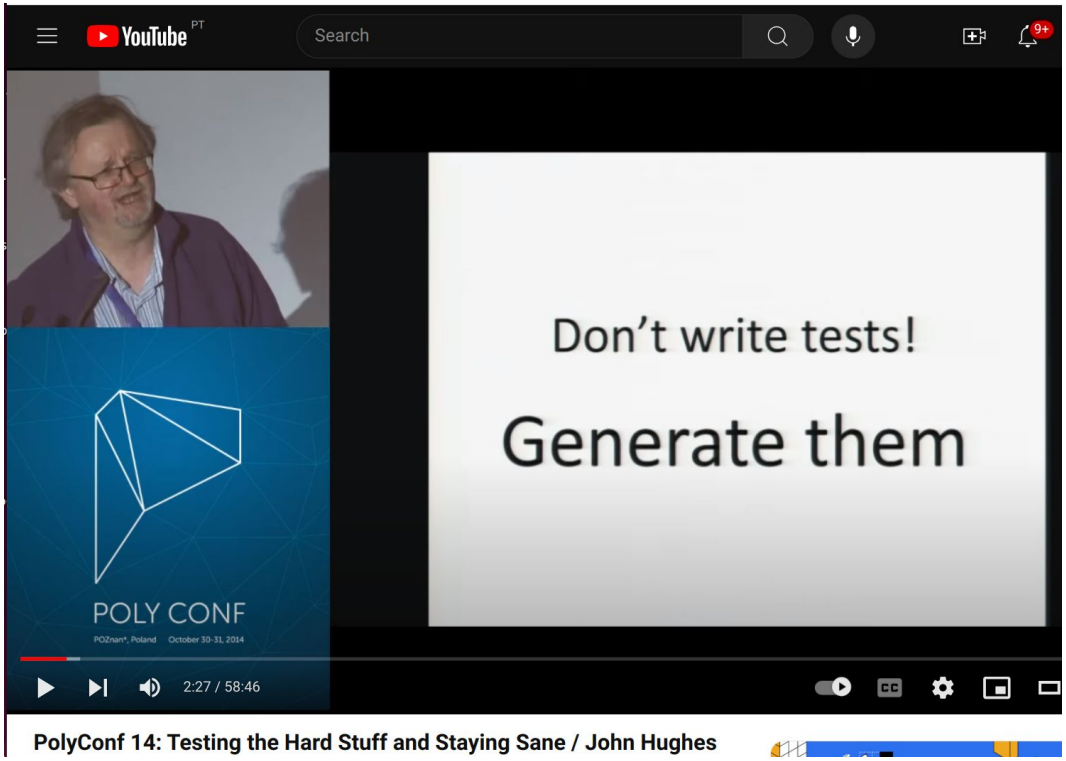
João Saraiva

2025/2026

Departamento de Informática
Universidade do Minho
Portugal

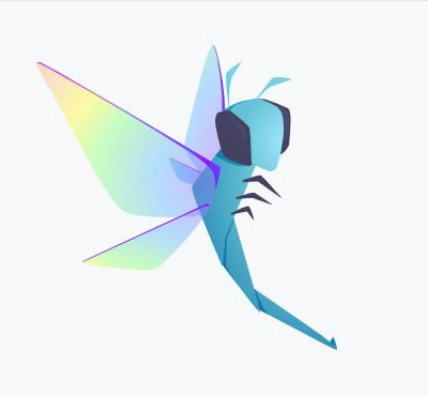
Don’t write tests!

Generate them



John Hughes - Don't Write Tests!

Automated Testing - Hypothesis



Hypothesis is an excellent Python library that allows expressing properties about programs and testing them with randomly-generated values.

Hypothesis Web Page

Installing

Hypothesis is available on PyPI as “hypothesis”. You can install it with:

```
pip install hypothesis
```

Automated Testing - Hypothesis

A test in *Hypothesis* consists of two parts:

- A function that looks like a normal test in your test framework of choice but with some additional arguments,
- A @given decorator that specifies how to provide those arguments.

Test Framework in Python - pytest

pytest: helps you write better programs

The `pytest` framework makes it easy to write small, readable tests, and can scale to support complex functional testing for applications and libraries.

`pytest` requires: Python 3.7+ or PyPy3.

PyPI package name: pytest

Example: (file **example.py**)

```
import pytest

def test_list_append_associative():
   x,y,z = [2] , [3,4,5],  [6,7]
   assert (x + y) + z == x + (y + z)
```

(tests need to start with test_)

Test Framework in Python - pytest

We can parameterized the (specific) arguments of the test case:

```
import pytest

def test_list_append_associative():
  x,y,z = [2] , [3,4,5],  [6,7]
  assert (x + y) + z == x + (y + z)

@pytest.mark.parametrize
   (("x","y","z"),[([1,7,12],[15,17], [27])
                  , ([2]    , []     , [6,7])
                  ]
   )
def test_list_append_associative_param(x,y,z):
    assert (x + y) + z == x + (y + z)
```

Test Framework in Python - pytest

```
saraiva@DELL:~$ python3 -m pytest example.py
============================= test session starts ==============================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: /home/saraiva
plugins: hypothesis-6.70.1
collected 3 items

example.py ...                                                           [100%]

============================== 3 passed in 0.01s ===============================
saraiva@DELL:~$
```

As expected, the 3 test cases passed.

Automated Testing - Hypothesis

Property-based Testing combines test case generation with property checking. In Hypothesis we use the @given decorator to use (pre-defined) strategies. In the next example, we use generators for integers to provide those arguments:

```
from hypothesis import given
from hypothesis.strategies import integers

@given(integers(),integers(),integers())
def test_add_associative(x,y,z):
    assert (x + y) + z == x + (y + z)
```

Automated Testing - Hypothesis

In the next example, we use predefined generators for **`lists`** and **`integers`** to provide those arguments:

```
from hypothesis import given
from hypothesis.strategies import lists, integers

@given(lists(integers()),lists(integers())
       ,lists(integers()))
def test_list_append_associative(x,y,z):
    assert (x + y) + z == x + (y + z)
```

Automated Testing - Hypothesis

```
from hypothesis import given
from hypothesis.strategies import *

@given(lists(integers()),lists(integers())
       ,lists(integers()))
def test_list_append_associative(x,y,z):
    assert (x + y) + z == x + (y + z)
```

Automated Testing - Hypothesis

```
saraiva@DELL:~$ python3 -m pytest example.py
============================= test session starts ==============================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: /home/saraiva
plugins: hypothesis-6.70.1
collected 1 item

example.py .                                                             [100%]

============================== 1 passed in 0.68s ===============================
saraiva@DELL:~$
```

- @given generates 100 (default value) random test cases.
- All test cases [100%] passed, thus 1 property passed.

- We can also produce several statistics (runtime, number of test cases, etc) with option:

--hypothesis-show-statistics

Automated Testing - Hypothesis

```
saraiva@DELL:~$ python3 -m pytest example.py --hypothesis-show-statistics
============================= test session starts ==============================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: /home/saraiva
plugins: hypothesis-6.70.1
collected 1 item

example.py .                                                             [100%]
============================ Hypothesis Statistics =============================

example.py::test_list_append_associative:

  - during reuse phase (0.01 seconds):
    - Typical runtimes: ~ 2ms, ~ 69% in data generation
    - 2 passing examples, 0 failing examples, 0 invalid examples

  - during generate phase (0.54 seconds):
    - Typical runtimes: 1-6 ms, ~ 84% in data generation
    - 98 passing examples, 0 failing examples, 7 invalid examples

  - Stopped because settings.max_examples=100


============================== 1 passed in 0.56s ===============================
saraiva@DELL:~$
```

Automated Testing - Hypothesis

Obviously we may define properties that fail.

```
from hypothesis import given
from hypothesis.strategies import integers

@given(integers(),integers())
def test_multiply_then_divide_is_the_same(x,y):
  assert (x * y) / y == x
```

Automated Testing - Hypothesis

```
saraiva@DELL:~/Teaching/2022-2023/ATS/Hypothesis/examples$ python3 -m pytest slides_fail.py --hypothesis-show-statistics
============================= test session starts ==============================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: /home/saraiva/Teaching/2022-2023/ATS/Hypothesis/examples
plugins: hypothesis-6.70.1
collected 1 item

slides_fail.py F                                                         [100%]

=================================== FAILURES ===================================
_____________________ test_multiply_then_divide_is_the_same _____________________

    @given(integers(),integers())
>   def test_multiply_then_divide_is_the_same(x,y):

slides_fail.py:6:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

x = 0, y = 0

    @given(integers(),integers())
    def test_multiply_then_divide_is_the_same(x,y):
>     assert (x * y) / y == x
E     ZeroDivisionError: division by zero

slides_fail.py:7: ZeroDivisionError
============================ Hypothesis Statistics =============================
slides_fail.py::test_multiply_then_divide_is_the_same:

  - during reuse phase (0.05 seconds):
    - Typical runtimes: 1-15 ms, ~ 28% in data generation
    - 0 passing examples, 10 failing examples, 0 invalid examples
    - Found 2 distinct errors in this phase

  - during shrink phase (0.08 seconds):
    - Typical runtimes: 0-1 ms, ~ 60% in data generation
    - 28 passing examples, 17 failing examples, 24 invalid examples
    - Tried 69 shrinks of which 0 were successful

  - Stopped because nothing left to do


=========================== short test summary info ============================
FAILED slides_fail.py::test_multiply_then_divide_is_the_same - ZeroDivisionError: division by zero
============================== 1 failed in 0.15s ===============================
```

Automated Testing - Hypothesis

A test/property properties may include several assertions.

```
from hypothesis import given
from hypothesis.strategies import integers

@given(integers(),integers())
def test_arithmetic_properties (x,y):
    assert x + y  == y + x
    assert x * y == y * x
    assert x * 1 == x
    assert x * 0 == 0
    assert x + 0 == x
    assert x + x == 2 * x
```

Automated Testing - Hypothesis

We may use @settings to change the number of test cases @given generates

```
from hypothesis import given , settings
from hypothesis.strategies import integers
  ~
@given(integers(),integers())
@settings(max_examples=500)
def test_arithmetic_properties (x,y):
    assert x + y == y + x
    assert x * y == y * x
    assert x * 1 == x
    assert x * 0 == 0
    assert x + 0 == x
    assert x + x == 2 * x
```

Automated Testing - Hypothesis

```
saraiva@DELL:~$ python3 -m pytest example.py --hypothesis-show-statistics
============================= test session starts ==============================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: /home/saraiva
plugins: hypothesis-6.70.1
collected 1 item

example.py .                                                             [100%]
========================== Hypothesis Statistics ===========================

example.py::test_arithmetic_properties:

  - during reuse phase (0.00 seconds):
    - Typical runtimes: ~ 1ms, ~ 69% in data generation
    - 1 passing examples, 0 failing examples, 0 invalid examples

  - during generate phase (0.50 seconds):
    - Typical runtimes: < 1ms, ~ 74% in data generation
    - 499 passing examples, 0 failing examples, 0 invalid examples

  - Stopped because settings.max_examples=500
```

Automated Testing - Hypothesis

Obviously, we can write **ill formed** properties that fail when executed with the random generated test cases.

```
@given(integers(),integers())
def test_arithmetic_properties (x,y):
    assert x + y == y + x
    assert x * y == y * x
    assert x * 1 == x
    assert x * 0 == 0
    assert x + 0 == x
    assert x + x == 2 * x
    assert x - y == y - x        # ??!
```

Ideas?

Automated Testing - Hypothesis

```
    @given(integers(),integers())
    def test_arithmetic_properties (x,y):
      assert x + y  == y + x
      assert x * y == y * x
      assert x * 1 == x
      assert x * 0 == 0
      assert x + 0 == x
>     assert x - y == y - x     # ??!
E     assert (0 - 1) == (1 - 0)

example.py:23: AssertionError
============================ Hypothesis Statistics ============================
example.py::test_arithmetic_properties:

  - during generate phase (0.03 seconds):
    - Typical runtimes: 0-13 ms, ~ 51% in data generation
    - 6 passing examples, 4 failing examples, 0 invalid examples
    - Found 1 distinct error in this phase

  - during shrink phase (0.05 seconds):
    - Typical runtimes: 0-2 ms, ~ 54% in data generation
    - 3 passing examples, 10 failing examples, 22 invalid examples
    - Tried 35 shrinks of which 12 were successful

  - Stopped because nothing left to do


========================== short test summary info ==========================
FAILED example.py::test_arithmetic_properties - assert (0 - 1) == (1 - 0)
============================== 1 failed in 0.10s ===============================
saraiva@DELL:~$
```

Automated Testing - Hypothesis

TODO: Introduce the previous example as “define a mutant” and not adding a new rule.

```
@given(integers(),integers())
def test_arithmetic_properties (x,y):
    assert x + y == y + x
    assert x * y == y * x
    assert x * 1 == x
    assert x * 0 == 0
    assert x + 0 == x
    assert x + x == 2 * x
    assert x - y == y - x       # ??!
```

Ideas?

Automated Testing - Hypothesis

Sometimes Hypothesis doesn't give you exactly the right sort of data we want. We can ignore these by aborting the test early using the assume assertion.

```
@given(integers(),integers())
@settings(max_examples=500)
def test_arithmetic_neg_numbers_property (x,y):
    assume (x < 0)
    assume (y < 0)
    assert x * y > 0
```

Automated Testing - Hypothesis

```
@given(i=st.integers().filter(lambda x: x % 2 == 0)
def test_even_filter (i: int) -> None:
    assert i % 2 == 0

@given(i=st.integers().map(lambda x: x * 2)
def test_even_map (i: int) -> None:
    assert i % 2 == 0
```

Automated Testing - Hypothesis

```
from typing import List

def bubble_sort(arr: List[int]) -> List[int]:
    for i in range(len(arr)):
        for j in range(0, len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
    return arr

@given(lists(integers()))
def test_sort_result_is_ordered(lst):
    sorted = bubble_sort(lst)
    for i in range(len(sorted) -1):
        assert sorted[i] <= sorted[i+1]
```

Automated Testing - Hypothesis

Just like in QuickCheck, we can define our own generators.

```
@composite
def gen_even_ints(draw):
    x = draw(integers())
    #y = draw (...)
    if (x < 0)
        return x * (-2)
    else
        return x * 2

@given(gen_even_ints())
def test_isEven(i):
    assert (x % 2) == 0
```

Automated Testing - Hypothesis

Exercises:

```
# 1- List of float:
# reverse (reverse l) == l



# 2- sum(reverse l) == sum(l)
```

Automated Testing - Hypothesis

Exercises:

```
# 1- reverse (reverse l) == l
@given(lists(floats()))
def test_list_reverse_twice(l: list[float]):
    assert l[::-1][::-1] == l

# 2- sum(reverse l) == sum(l)
@given(lists(integers()))
def test_list_reverse_sum(l: list[int]):
    assert sum(l[::-1]) == sum(l)
```

Automated Testing - Hypothesis

```
# 3-reverse(l1 + l2) == reverse l2 + reverse l1




# 4- delete x (x:l) == l
```

Automated Testing - Hypothesis

```
# 3-reverse(l1 + l2) == reverse l2 + reverse l1
@given(lists(integers()), lists(integers()))
def test_list_reverse_concat(l1, l2):
  assert (l1 + l2)[::-1] == l2[::-1] + l1[::-1]

# 4- delete x (x:l) == l
@given(integers(), lists(integers()))
def test_list_remove(x: int, l: list[int]):
    l2 = [x] + l
    l2.remove(x)
```

Automated Testing - Hypothesis

TODO: Shrinking

The last aspect of property-based testing that this project will focus on is counterexample minimization. Just like generating data that satisfy preconditions directly can be exponentially more efficient than generating arbitrary data and filtering them, the same is true for minimizing reported counterexamples that need to satisfy the same invariants. A related problem is that of counterexample deduplication; when testing a piece of software for a non-trivial amount of time, usually many different bugs are discovered. Being able to characterize which counterexamples correspond to the same underlying problem in the code helps both reduce the effort on the end user (that otherwise has to sort through numerous instances of the same issue), and improve the efficiency of testing (by avoiding input data that exhibit identical erroneous behavior). Existing techniques in the coverage-based fuzzing world, like comparing stack trace hashes or coverage pro-files, fall way short of discriminating instances of the same error correctly.